

Aula 01: Método de Pólya

Como resolver problemas?

Maurício da Silva Pires
mspires@id.uff.br

TCC00326 - Programação de computadores
Niterói, Rio de Janeiro

Sejam bem-vindos a um curso de programação de computadores!

- ⌋ Antes de começarmos a falar sobre o que mais nos interessa, precisamos desmistificar alguns pontos
- ⌋ **Programar não é difícil** - A dificuldade geral das pessoas está em entender que o computador é uma ferramenta excelente em seguir ordens
 - * Existem ordens certas e ordens erradas?
 - * Existe uma única maneira de se resolver uma tarefa?
 - * Como as ordens impactam o resultado de uma computação?

- ⌈ A existência de ordens certas e erradas é, em certa medida, algo relativo, ou seja, temos que destacar algumas nuances
 - * As ordens dadas a um computador possuem um formato específico. Logo, não respeitar o formato caracterizaria uma ordem como **errada**
 - * Em contrapartida, como não há uma única forma de se resolver um problema, o certo e errado no contexto pode se referir ao resultado final, isto é, se os resultados são iguais ou não para duas estratégias de resolução de um problema
- ⌈ Isso já nos instrui sobre a existência de dois tipos de erros distintos que podemos encontrar
 - * Erro do formato da ordem
 - * Erro do encadeamento das ordens

Resolução de problemas

- ⌋ Em todos os campos de estudo estamos constantemente refletindo sobre maneiras de resolver problemas.
 - * Fenômenos e situações são observados, analisados e estudados sob diversos aspectos.
- ⌋ Por mais que tenhamos a impressão de que o fazemos de forma “natural”, instintiva, na verdade estamos realizando uma série de processos mentais complexos
 - * Existe um histórico de problemas previamente resolvidos e de que temos conhecimento
 - * Analisamos as características do nosso problema e começamos a tomar diversas decisões com base nessas análises

Um exemplo muito intuitivo

- ⌋ Após ler o exemplo a seguir, você provavelmente vai se questionar se este é de fato um curso de programação de computadores, mas peço um pouco de paciência
- ⌋ Imagine que você está diante de uma porta e deseja abri-la. O que você faz?
 - * Estendo a mão, toco a maçaneta, giro a maçaneta e puxo ou empurro a porta dependendo se vejo as dobradiças da porta ou não
 - * Embora pareça algo óbvio, percebe quantas coisas diferentes foram feitas nesse simples ato de abrir uma porta?
 - * E se, mesmo fazendo isso, a porta não se abre? O que pode ter acontecido?

Um exemplo muito intuitivo

- ⌋ Estamos tão acostumados a executar algumas resoluções de problemas tantas vezes ao dia a ponto de abstrairmos o que está acontecendo
- ⌋ Por outro lado, as máquinas não possuem essa capacidade de abstração, afinal, o núcleo de processamento do computador (diferentemente do cérebro humano) possui um conjunto limitado de coisas que pode fazer
- ⌋ Aqui vem a primeira dica para quem está aprendendo a programar: **o óbvio para você não é para a máquina**
 - ✳ Ao longo do curso, eu provavelmente farei perguntas cujas respostas parecem óbvias e, na maioria das vezes, de fato esta será a resposta correta

George Pólya (1887-1985)

- ☾ Matemático húngaro que trabalhou em universidades de grande prestígio, como Stanford e Brown
 - * Sua produção acadêmica é vasta e se estende por áreas como séries, teoria dos números, análise matemática, geometria, álgebra, combinatória e probabilidade
 - * É autor/coautor de grandes obras na área de resolução de problemas
 - * Nunca se esqueçam que a matemática nasceu da filosofia
- ☾ Em 1945, Pólya publica um volume curto chamado *How To Solve It*, que abordava um guia para resolver problemas, que, embora pensado para a Matemática, pode ser aplicado em outras áreas

How to solve it (resumidamente) ---

☾ O processo proposto por Pólya consiste nas seguintes etapas:

1. **Compreender o problema**, destacando o que se tem disponível e o que se deseja obter
2. **Elaborar um plano** a partir das informações disponíveis
3. **Executar o plano** proposto
4. **Revisar o resultado** em busca de melhorias

☾ Em casos que não é possível resolver o problema, o que fazer?

- * Buscar problemas mais simples que possuam solução
- * Buscar problemas semelhantes e que possuam solução

Resolvendo computacionalmente

- ⌋ Em termos de resolver problemas usando o computador, o guia de Pólya assume a seguinte nomenclatura
 1. **Levantamento de requisitos** é a etapa de compreensão
 2. **Prototipação** é a elaboração de um plano para a resolução
 3. **Codificação** é a etapa de execução do plano
 4. **Testagem** é a etapa de revisão dos resultados

- ⌋ As etapas adicionais também possuem nomenclaturas específicas
 - ✱ **Decomposição** é estratégia de usar problemas mais simples
 - ✱ **Analogia** é a estratégia de usar problemas semelhantes

Levantamento de requisitos

- ☾ O lema desta etapa é: **Analisar, entender e anotar!**
- ☾ Exemplos de perguntas a serem respondidas nesta etapa
 - * Quais informações devem ser inseridas pelos usuários?
 - * Quais são as respostas esperadas na resolução do problema?
 - * Que tipo de informações devem ser computadas a partir das informações passadas pelos usuários?
 - * Os dados manipulados possuem algum limite?
 - * Qual é o formato das respostas dadas pelo programa?
- ☾ Caso existam exceções e cenários especiais de funcionamento do programa, estes devem ser definidos nesta etapa

Prototipação de uma solução

- ⌋ A etapa de prototipação é conhecida por ser a etapa de construção de um algoritmo
- ⌋ **Algoritmo** é um conceito que existe muito antes da própria computação
 - * Define-se algoritmo como uma **sequência finita de passos simples e precisos para executar uma tarefa ou resolver um problema**
 - * Precisão é o mesmo que “sem ambiguidade”
- ⌋ Todo algoritmo é composto por 3 partes essenciais
 - * **Entrada:** Os dados iniciais que o algoritmo recebe
 - * **Processamento:** As instruções e cálculos executados
 - * **Saída:** O resultado final gerado após o processamento

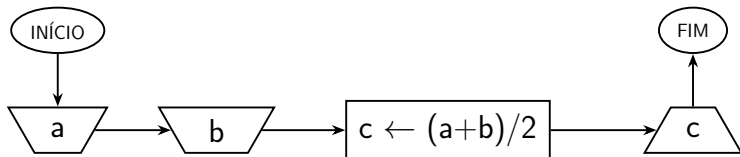
Prototipação por pseudocódigo

- ⌋ A estratégia consiste em usar comandos similares aos encontrados em linguagens de programação, mas na linguagem nativa do usuário
- ⌋ Vejam o guia de pseudocódigo disponibilizado em aula
- ⌋ Exemplo: algoritmo para calcular a média simples de dois números.

```
Início
    ler(a)                # Entrada
    ler(b)                # Entrada
    c <- (a+b)/2          # Atribuição
    exibir(c)             # Saída
Fim
```

Prototipação por fluxograma

- ⌋ A estratégia consiste em usar formas geométricas diferentes para codificar tipos de ações (instruções). O fluxo de execução do algoritmo é definido por meio de setas
- ⌋ Vejam o guia de fluxograma disponibilizado em aula
- ⌋ Exemplo: algoritmo para calcular a média simples de dois números.



Codificação do programa

- ☾ Uma vez que sabemos quais instruções devem ser utilizadas, passamos para a parte de codificação
 - * A tradução do algoritmo para uma linguagem de programação
 - * A maior dificuldade é a elaboração do algoritmo
- ☾ Em nossa disciplina, iremos utilizar a linguagem Python
 - * A sintaxe do Python é semelhante à sintaxe do inglês, o que facilita a compreensão e a legibilidade do código
 - * A linguagem é *open-source*, o que significa que a sua documentação é de fácil acesso para desenvolvedores
 - * Além da documentação oficial, há uma ampla gama de recursos sobre o Python, incluindo materiais didáticos e conteúdo digital gratuito

Testagem do código produzido

- ⌋ Os testes são conduzidos para encontrar dois tipos de erros principais.
 - * Erros de sintaxe oriundos da etapa de codificação (instruções escritas erradas)
 - * Erros de prototipação, ou seja, erros no encadeamento lógico das instruções gerando resultados inesperados
- ⌋ Em geral, os testes são realizados com um conjunto de entradas variadas
 - * Boa parte dessas entradas (senão todas) tem o resultado de sua computação conhecida previamente
 - * Os resultados obtidos são comparados e possíveis erros de prototipagem podem ser detectados

Analizando um problema

- ☾ Um palíndromo é uma palavra/frase que é idêntica se lida da esquerda para a direita quanto da direita para a esquerda
 - * “ovo” e “arara” são palavras que são palíndromos
 - * “ame o poema” é um palíndromo
 - * “a mala na lama” é um palíndromo
 - * “1010101” é um palíndromo
- ☾ Seu problema é projetar um algoritmo que consiga determinar se uma palavra ou uma frase é ou não um palíndromo

Levantamento de requisitos

☾ Sobre a entrada

- * O usuário entra com uma palavra ou uma frase
- * Existe a possibilidade de as letras serem maiúsculas ou minúsculas, além de poder ter acentos ou símbolos especiais
- * Precisamos lidar com os espaços em branco, os acentos e os símbolos especiais

☾ Sobre a saída

- * Imprimir na tela “É palíndromo” ou “Não é palíndromo”

Prototipação de solução

- ⌋ A solução deste problema precisa contemplar duas etapas principais
 - * A “normalização” da entrada, que é a remoção dos espaços em branco, símbolos especiais, acentos e cedilhas
 - * A decisão de que a cadeia de caracteres é um palíndromo ou não
- ⌋ E como podemos decidir se uma cadeia de caracteres é um palíndromo?

Prototipação de solução

- ⌋ A solução deste problema precisa contemplar duas etapas principais
 - * A “normalização” da entrada, que é a remoção dos espaços em branco, símbolos especiais, acentos e cedilhas
 - * A decisão de que a cadeia de caracteres é um palíndromo ou não
- ⌋ E como podemos decidir se uma cadeia de caracteres é um palíndromo?
 - * Fazer uma cópia da cadeia normalizada, inverter a cópia e comparar com a cadeia original
 - * Se todas as letras estiverem no mesmo lugar, então é um **palíndromo**

Prototipação de solução

- ⌈ A solução deste problema precisa contemplar duas etapas principais
 - * A “normalização” da entrada, que é a remoção dos espaços em branco, símbolos especiais, acentos e cedilhas
 - * A decisão de que a cadeia de caracteres é um palíndromo ou não
- ⌈ E como podemos decidir se uma cadeia de caracteres é um palíndromo?
 - * Enquanto a cadeia tiver tamanho maior ou igual a 2, compare a primeira letra com a última. Se forem iguais, remova-as. Se forem diferentes, pare a repetição
 - * Se após a repetição a cadeia tiver tamanho 1 ou 0, então é um **palíndromo**

Codificação da solução

☾ Uma possível codificação para a solução 1 do problema

```
def normaliza(frase):  
    ...  
  
texto = input("Entre com um a frase/palavra: ")  
texto = normaliza(texto)  
inverso = texto[::-1]  
for i in range(len(texto)):  
    if texto[i] != inverso[i]:  
        print("Não é um palíndromo")  
        break  
else:  
    print("É um palíndromo")
```

Codificação da solução

☞ Uma possível codificação para a solução 2 do problema

```
def normaliza(frase):  
    ...  
  
texto = input("Entre com um a frase/palavra: ")  
texto = normaliza(texto)  
while len(texto) > 1:  
    if texto[0] != texto[-1]:  
        print("Não é palíndromo")  
        break  
    texto = texto[1:-1]  
else:  
    print("É um palíndromo")
```

☾ Dos códigos anteriores, temos

- * **normaliza**(texto) é uma função auxiliar, ou seja, função criada pelo programador (Veremos como fazer isso na Aula 06)
- * **inverso = texto[::-1]** é uma instrução que inverte uma cadeia de caracteres (Veremos como fazer isso na Aula 09)
- * **texto = texto[1:-1]** é uma estratégia para remover o primeiro e o último caracteres de uma cadeia de caracteres (Veremos como fazer isso na Aula 09)
- * **for** e **while** são estruturas de repetição (Veremos este assunto na próxima aula)

☾ Antes de pensar em codificar algo, foque sua energia em entender como elaborar um plano para lidar com o problema

Verificação do resultado

- ﴿ Inseriremos entradas que conhecemos as respostas
 - * **Casos de testes positivos** (palíndromos): “ovo”, “arara”, “A mala na lama”, “ame o poema”
 - * **Casos de testes negativos** (não palíndromos): “coruja”, “computador”, “Uma noite de verão”, “Rock in Rio”
 - * O tamanho do conjunto de testes pode melhorar a qualidade da verificação
- ﴿ Podemos empregar testes de mesa para ajudar na análise
 - * Teste de mesa é uma técnica manual para testar a lógica de um algoritmo ou código sem usar o computador
 - * Você simula o funcionamento do programa usando papel, caneta e uma tabela

Recapitulando

- ⌈ O Método de Pólya para resolução de problemas consiste em (1) entender o problema, (2) elaborar um plano para resolvê-lo, (3) executar o plano e (4) verificar os resultados obtidos
- ⌈ Algoritmos são sequências finitas de instruções a serem executadas e estão associados à segunda etapa do Método de Pólya
- ⌈ Embora os algoritmos não dependam de linguagens de programação, a terceira etapa do Método de Pólya, na Computação, se associa à geração do programa a ser executado. É aqui que o conhecimento da linguagem é indispensável

